



Git Flow

- 일관성 유지 : 팀 전체가 동일한 규칙 사용
- 정기적인 회고를 통한 프로세스 개선
- 문서화를 통한 지식 공유

1. Issues / Milestone / Projects

<https://docs.github.com/en/issues/using-labels-and-milestones-to-track-work/about-milestones>

- **Issues** : 라벨
- **Milestone** : PR 할때 연결이 가능, 진행상황 추적 가능
- **Projects** : To-Do, 일정 관리, 진행과정

2. Squash & Merge

- 여러 개의 커밋을 하나로 합쳐서 깔끔한 커밋 히스토리를 유지하는 머지 방식
- 커밋 메시지 꼭 적기 → Squash 시 의미 있는 커밋 메시지 작성



Changes approved

1 approving review by reviewers with write access.



1 approval >



No conflicts with base branch

Merging can be performed automatically.

Squash and merge



You can also merge this with the command line.

[View command line instructions.](#)

1 개발 및 커밋

기능 개발 후 커밋

```
git commit -m "커밋 메시지"
```

깃모지 사용일 경우

```
git add .
```

```
gitmoji -c
```

feat 검색 → ✨ 선택

띄어쓰기 후 소문자로 시작

feat: 사용자 인증 API 개발

Enter 후 상세 내용 작성: - JWT 토큰 기반 인증 시스템 구현

원격 브랜치에 푸시

```
git push origin feature/branch
```

2 Pull Request 생성

1. GitHub 사이트 접속
2. PR 생성 버튼 클릭 (**Push** 후 자동으로 나타남)
3. PR 설명 작성 (상세할수록 좋음)
4. 코드 리뷰 한 명이 **Approve** 누르면 squash and merge로 PR 제출

3 코드 리뷰

- 1명 이상의 승인 필요 **Approve**
- 리뷰 피드백 반영 후 수정 사항 **Push**

4 Squash and Merge 실행

1. **Squash and Merge** 선택
2. **중요:** 제목에서 **(#5)** 같은 PR 번호 제거
3. 커밋 메시지를 본인이 개발한 내용으로 수정
(이전 커밋 내역 삭제 후 현재 PR 반영 커밋만 남김)
4. **Merge 완료** → **develop** 브랜치에 반영

Merge 후 작업

5 팀원들에게 알림

- 머지 완료 알림

6 로컬 브랜치 업데이트

```
# develop 브랜치로 이동
git checkout develop

# 최신 develop 브랜치 내용 가져오기
git pull origin develop

# 본인 기능 브랜치로 이동
git checkout feature/user

# develop 브랜치 내용을 본인 브랜치에 머지
git merge develop
```

7 충돌 해결

- 머지 과정에서 충돌 발생 시 즉시 해결
- 충돌 해결 후 커밋

완료 및 다음 작업

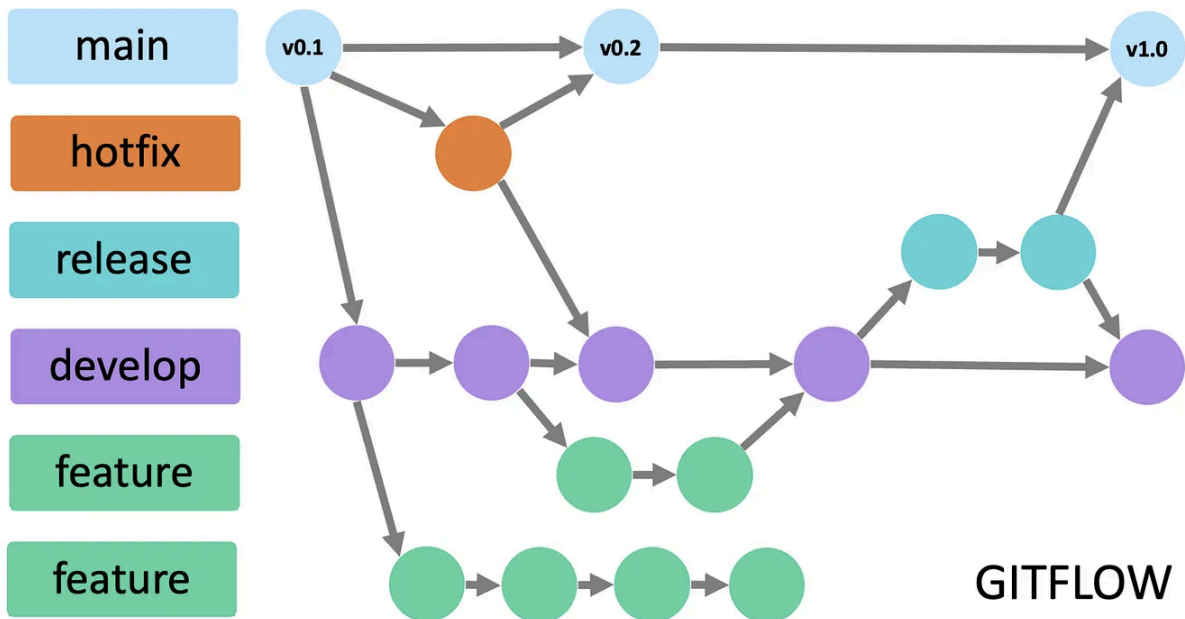
모든 과정이 완료되면 새로운 기능 개발 시작 가능

주의사항

- PR 제목에서 번호 제거 필수
- 커밋 메시지는 개발 내용을 명확히 작성
- 충돌은 머지 과정에서 반드시 해결할 것
- 팀원들에게 머지 완료 알림 필수

3. git flow 브랜치 전략

- Git Flow의 브랜치 전략과 Squash & Merge의 머지 방식을 조합하여 체계적이고 깔끔한 Git 워크플로우를 구축



- **hotfix** : develop에서 생성
- project 특성상 release 브랜치를 사용하지 않습니다.

🌳 브랜치 구조 (Git Flow)

Git Flow 브랜치 구조

```

main (production)
├── develop (개발 통합)
│   ├── feature/member
│   ├── feature/payment
│   ├── feature/admin
│   └── ...
├── docs/api-documentation (문서 작업)
└── hotfix/critical-bug (긴급 수정)
  
```

- 자동화 도구 활용 (GitHub Actions, GitLab CI 등)

1 기능 개발 시작

```
# develop 브랜치에서 feature 브랜치 생성
git checkout develop
git pull origin develop
git checkout -b feature/member
```

2 개발 및 커밋

```
# 기능 개발 (여러 번의 작은 커밋들)
git add .
git commit -m "로그인 UI 구현"
git commit -m "로그인 API 연동"
git commit -m "에러 처리 로직 추가"
git commit -m "테스트 코드 작성"
```

```
# 깃모지 사용
git add .
gitmoji -c
```

```
feat 검색 → ✨ 선택
띄어쓰기 후 소문자로 시작
feat: 사용자 인증 API 개발
Enter 후 상세 내용 작성: - JWT 토큰 기반 인증 시스템 구현
```

```
# 원격 브랜치에 푸시
git push origin feature/member
```

▼ 3 Pull Request 생성 (feature → develop)

1. GitHub에서 PR 생성
2. **Base:** `develop`, **Compare:** `feature/member`
3. 상세한 PR 설명 작성
4. PR 제출

▼ 4 코드 리뷰 및 Squash & Merge

1. 1명 이상의 승인 받기
2. **Squash and Merge** 선택

3. **중요:** 제목에서 (#번호) 제거
 4. 커밋 메시지 수정: "사용자 로그인 기능 구현"
 5. **Develop** 브랜치에 머지 완료
-

각 Branch의 Merge 전략

feature → develop

- 방식: Squash & Merge
- 목적: 기능 단위로 커밋 정리
- 결과: 하나의 깔끔한 커밋

▼ Feature 브랜치 작업

```
# 1. 최신 develop 반영
git checkout develop
git pull origin develop
git checkout feature/user
git merge develop

# 2. 충돌 해결 (있다면)# 충돌 파일 수정 후
git add .
git commit -m "develop 브랜치 머지"

# 3. 개발 계속...
```

develop → main

- 방식: Merge Commit 또는 Squash & Merge
- 목적: 프로덕션 배포
- 결과: 메인 브랜치 생성

hotfix → main & develop

- 방식: Merge Commit
- 목적: 긴급 수정사항 반영
- 결과: 양쪽 브랜치 모두 업데이트

▼ hotfix 브랜치 작업

```
# 1. Main에서 Hotfix 브랜치 생성
git checkout main
git checkout -b hotfix/critical-security-fix

# 2. 긴급 수정사항 적용
git commit -m "보안 취약점 수정"

# 3. Main과 Develop 모두에 머지
git checkout main
git merge hotfix/critical-security-fix
git tag v1.2.1

git checkout develop
git merge hotfix/critical-security-fix
```

▼ 🕒 팀원 알림

- 머지 완료 알림
- 배포 일정 공유 (해당하는 경우)

▼ 🌟 코드 품질

- 체계적인 브랜치 관리로 안정적인 개발 환경
- **Squash & Merge**로 깔끔한 커밋 히스토리

▼ 📞 추적 가능성

- 기능 단위 커밋으로 쉬운 코드 추적

▼ 📦 효율성

- 병렬 개발 지원
- 쉬운 롤백 가능

▼ 🕒 규칙 준수

- 브랜치 명명 규칙 일관성 유지
- 커밋 메시지 품질 관리
- **PR 설명** 상세히 작성

▼ ⚠️ 충돌 관리

- 정기적인 **develop** 머지로 충돌 최소화
- 충돌 발생 시 즉시 해결

▼ 👤 팀 협업

- 머지 완료 알림 필수
 - 코드 리뷰 적극 참여
-